

EQUAZIONI DI RICORRENZA

PIETRO DI LENA

DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
UNIVERSITÀ DI BOLOGNA

ALGORITMI E STRUTTURE DI DATI
ANNO ACCADEMICO 2024/2025



INTRODUZIONE

- Una **equazione di ricorrenza** descrive ogni elemento in una sequenza in termini degli elementi precedenti

- Abbiamo già visto l'equazione di ricorrenza di Fibonacci

$$T(n) = \begin{cases} 1 & n \leq 2 \\ T(n-1) + T(n-2) + 1 & n > 2 \end{cases}$$

- Vogliamo determinare l'ordine di crescita delle equazioni di ricorrenza

- Determinare la crescita asintotica degli **algoritmi ricorsivi**

- Vedremo due metodi per risolvere equazioni di ricorrenza

- **Metodo dell'iterazione**
 - **Master Theorem**
 - ~~■ Metodo della sostituzione~~
 - ~~■ Metodo dell'albero di ricorsione~~

NOZIONI PRELIMINARI

- Le equazioni di ricorrenza di algoritmi ricorsivi possono essere del tipo

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ 2T(\lfloor n/3 \rfloor) + O(n) & n > 1 \end{cases}$$

- Tipicamente sostituiamo la notazione asintotica con **espressioni positive**

$$T(n) = \begin{cases} d & n = 1 \\ 2T(\lfloor n/3 \rfloor) + cn & n > 1 \end{cases}$$

- Tipicamente utilizziamo la costante 1 invece di costanti simboliche

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(\lfloor n/3 \rfloor) + 1 \cdot n & n > 1 \end{cases}$$

- Il comportamento asintotico non è influenzato da **arrotondamenti**

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n/3) + n & n > 1 \end{cases}$$

METODO DELL'ITERAZIONE

- Il **metodo dell'iterazione** è un approccio di tipo *brute force*
- Idea: sostituiamo iterativamente la parte ricorsiva nell'equazione finché non appare uno **schema ricorsivo** legato al **passo di iterazione**
- Esempio. Consideriamo la ricorrenza $T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + c & n > 1 \end{cases}$

$$T(n) = T(n/2) + c \quad \text{passo 1}$$

$$= T(n/4) + c + c \quad \text{passo 2}$$

$$= T(n/8) + c + c + c \quad \text{passo 3}$$

...

$$= T(n/2^i) + c \cdot i \quad \text{passo } i$$

La ricorsione termina quando $n/2^i = 1 \implies i = \log_2 n$. Quindi

$$T(n) = T(1) + c \cdot \log_2 n = 1 + c \cdot \log_2 n = \Theta(\log n)$$

Otteniamo lo stesso risultato se sostituiamo $c = 1$

ESEMPIO: METODO DELL'ITERAZIONE

- Consideriamo la ricorrenza $T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + n & n > 1 \end{cases}$

$$T(n) = T(n/2) + n \quad \text{passo 1}$$

$$= T(n/4) + n/2 + n \quad \text{passo 2}$$

$$= T(n/8) + n/4 + n/2 + n \quad \text{passo 3}$$

...

$$= T(n/2^i) + n \sum_{k=0}^{i-1} 1/2^k \quad \text{passo } i$$

$$= T(n/2^i) + n \frac{1/2^i - 1}{1/2 - 1}$$

$$= T(n/2^i) + 2n(1 - 1/2^i)$$

La ricorsione termina quando $n/2^i = 1 \implies i = \log_2 n$. Quindi

$$T(n) = T(1) + 2n(1 - 1/n) = 1 + 2n - 2 = 2n - 1 = \Theta(n)$$

ESEMPIO: METODO DELL'ITERAZIONE

- Consideriamo la ricorrenza $T(n) = \begin{cases} 1 & n = 1 \\ 3T(n/4) + n^2 & n > 1 \end{cases}$

$$T(n) = 3T(n/4) + n^2 \quad \text{passo 1}$$

$$= 3^2 T(n/4^2) + 3(n/4)^2 + n^2 \quad \text{passo 2}$$

$$= 3^3 T(n/4^3) + 3^2(n/4^2)^2 + 3(n/4)^2 + n^2 \quad \text{passo 3}$$

...

$$= 3^i T(n/4^i) + n^2 \sum_{k=0}^{i-1} (3/16)^k \quad \text{passo } i$$

$$= 3^i T(n/4^i) + n^2 \frac{(3/16)^i - 1}{3/16 - 1}$$

La ricorsione termina quando $n/4^i = 1 \implies i = \log_4 n$. Quindi

$$T(n) = 3^{\log_4 n} \cdot T(1) + \frac{16}{13} n^2 (1 - 3/n^2) = n^{\log_4 3} + \frac{16}{13} n^2 - \frac{48}{13} = \Theta(n^2)$$

MASTER THEOREM

- Il **Master Theorem** è un approccio per risolvere ricorrenze della forma

$$T(n) = aT(n/b) + f(n)$$

con $a \geq 1$ e $b > 1$ **costanti** e $f(n)$ **asintoticamente positiva**

- Equazioni di ricorrenza di algoritmi che
 - dividono un problema di dimensione n in $a \geq 1$ sottoproblemi
 - tutti i sottoproblemi hanno dimensione n/b , con $b > 1$
 - il costo di ogni chiamata ricorsiva è dato da $f(n)$
- Non può essere applicato a tutte le possibili ricorrenze
 - Non può essere applicato all'equazione di ricorrenza di Fibonacci

MATHER THEOREM

Theorem (Master Theorem)

Si consideri la seguente equazione di ricorrenza

$$T(n) = \begin{cases} d & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

dove $a \geq 1$, $b > 1$, d costante e $f(n)$ è una funzione di costo. Allora

- 1** *Se $f(n) = O(n^{\log_b a - \epsilon})$ per qualche $\epsilon > 0$ allora $T(n) = \Theta(n^{\log_b a})$*
- 2** *Se $f(n) = \Theta(n^{\log_b a})$ allora $T(n) = \Theta(n^{\log_b a} \log n)$*
- 3** *Se $f(n) = \Omega(n^{\log_b a + \epsilon})$ per qualche $\epsilon > 0$, e se $a \cdot f(n/b) \leq c \cdot f(n)$ per qualche costante $c < 1$ e per tutti gli n sufficientemente grandi, allora $T(n) = \Theta(f(n))$*

N.B. In tutti e tre i casi confrontiamo $f(n)$ con $n^{\log_b a}$

MATHER THEOREM: VERSIONE SEMPLIFICATA

Theorem (Master Theorem)

Si consideri la seguente equazione di ricorrenza

$$T(n) = \begin{cases} d & n = 1 \\ aT(n/b) + cn^\beta & n > 1 \end{cases}$$

dove $a \geq 1$, $b > 1$ e c, d costanti. Sia $\alpha = \log_b a = \frac{\log a}{\log b}$. Allora

- 1** *Se $\alpha > \beta$ allora $T(n) = \Theta(n^\alpha)$*
- 2** *Se $\alpha = \beta$ allora $T(n) = \Theta(n^\alpha \log n)$*
- 3** *Se $\alpha < \beta$ allora $T(n) = \Theta(n^\beta)$*

N.B. Meno generale della versione precedente: ammette solo funzioni $f(n)$ della forma cn^β

ESEMPI: MASTER THEOREM

1 Risolvere col Master Theorem $T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + c & n > 1 \end{cases}$

Abbiamo $a = 1, b = 2, \alpha = \log_b a = \log_2 1 = 0, \beta = 0$

$$\alpha = \beta \Rightarrow T(n) = \Theta(n^\alpha \log n) = \Theta(\log n)$$

2 Risolvere col Master Theorem $T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + n & n > 1 \end{cases}$

Abbiamo $a = 1, b = 2, \alpha = \log_b a = \log_2 1 = 0, \beta = 1$

$$\alpha < \beta \Rightarrow T(n) = \Theta(n^\beta) = \Theta(n)$$

3 Risolvere col Master Theorem $T(n) = \begin{cases} 1 & n = 1 \\ 3T(n/4) + n^2 & n > 1 \end{cases}$

Abbiamo $a = 3, b = 4, \alpha = \log_4 3 \approx 0.79, \beta = 2$

$$\alpha < \beta \Rightarrow T(n) = \Theta(n^\beta) = \Theta(n^2)$$

ESEMPIO: RICORRENZA DELLA RICERCA BINARIA

Cercare la posizione di un valore all'interno di un array ordinato (-1 se non trovato)

```
1: function BINSEARCH(ARRAY  $A[1 \cdots n]$ , INT  $x$ , INT  $i$ , INT  $j$ )  $\rightarrow$  INT
2:   if  $i > j$  then
3:     return  $-1$ 
4:   else
5:      $m = (i + j) / 2$   $\triangleright$  Divisione intera
6:     if  $A[m] == x$  then
7:       return  $m$ 
8:     else if  $A[m] > x$  then
9:       return SEARCH( $A, x, i, m - 1$ )
10:    else
11:      return SEARCH( $A, x, m + 1, j$ )
```

- La prima chiamata è invocata con parametri SEARCH($A, x, 1, n$)
- **Caso ottimo** (x è nel centro dell'array): $O(1)$
- **Caso pessimo** (x non è nell'array): ?

ESEMPIO: RICORRENZA DELLA RICERCA BINARIA

```
1: function BINSEARCH(ARRAY  $A[1 \cdots n]$ , INT  $x$ , INT  $i$ , INT  $j$ )  $\rightarrow$  INT
2:   if  $i > j$  then
3:     return  $-1$ 
4:   else
5:      $m = (i + j)/2$   $\triangleright$  Divisione intera
6:     if  $A[m] == x$  then
7:       return  $m$ 
8:     else if  $A[m] > x$  then
9:       return SEARCH( $A, x, i, m - 1$ )
10:    else
11:      return SEARCH( $A, x, m + 1, j$ )
```

- Possiamo estrarre la funzione di ricorrenza dallo pseudocodice

$$T(n) = \begin{cases} 1 & n = 0 \text{ (costo costante se spazio di ricerca è 0)} \\ T(n/2) + 1 & n > 0 \text{ (costo costante + ricerca su 1/2 spazio)} \end{cases}$$

- Soluzione col Master Theorem:

$$\alpha = 0, \beta = 0 \implies T(n) = \Theta(n^0 \log n) = \Theta(\log n)$$

LIMITE STRETTO PER LA RICORRENZA DI FIBONACCI

- Abbiamo dimostrato che la ricorrenza di Fibonacci è limitata da

$$T(n) = \Omega(\sqrt{2}^n) \approx \Omega(1.41^n) \text{ e } T(n) = O(2^n)$$

- Possiamo trovare un limite più stretto?
 - Difficile con il metodo iterativo
 - Non possiamo applicare il Master Theorem

Teorema

Sia $T(n)$ l'equazione di ricorrenza di Fibonacci. Allora, $T(n) = 2F_n - 1$
(Dimostrazione per induzione)

- Dal Teorema sopra abbiamo che $T(n) = \Theta(2F_n - 1) = \Theta(F_n)$
- L' n -esimo numero di Fibonacci è definito da $F_n = \frac{1}{\sqrt{5}} \left(\phi^n - \hat{\phi}^n \right)$

$$\phi = \frac{1+\sqrt{5}}{2} \approx 1.618 \text{ e } \hat{\phi} = \frac{1-\sqrt{5}}{2} \approx -0.618$$

- Concludiamo che $T(n) = \Theta(\phi^n) \approx \Theta(1.62^n)$